LAPORAN PRAKTIKUM DATA STORAGE (1)
PEMROGRAMAN PERANGKAT BERGERAK (CCK3AAB3)
WEEK 09 ["CONSUME REST API SUPABASE DENGAN FLUTTER"]

Nama Mahasiswa	:	**Haryanto Wifakul Azmi**
NIM	:	**103122400037**
Kelas	:	SE-08-02
Dosen Pengampu	:	Yudha Islami Sulistya, S.Kom., M.Cs. [YIS]
Program Studi	:	S1 Software Engineering (Rekayasa Perangkat Lunak)
Institusi	:	Fakultas Informatika - Telkom University Purwokerto

A. TUJUAN & SOAL PRAKTIKUM

Pada Materi supabase consume, silahkan di kerjakan step by step dan hasilnya dalam bentuk PDF.

B. DASAR TEORI & ARSITEKTUR SISTEM

Aplikasi dirancang menggunakan arsitektur Simplified MVC dengan Service Layer yang rapi untuk menjaga prinsip separation of concerns (pemisahan tanggung jawab), terdiri dari:

- **1. Presentation Layer (lib/pages/todo_page.dart):** Menyediakan antarmuka visual (UI) pengguna, merender daftar tugas, menangani interaksi pengguna (tap, toggle, dialog tambah data, dan delete), serta merebuild UI berdasarkan state controller.
- **2. Controller Layer (lib/controllers/todo_controller.dart):** Menjadi penghubung antara UI dan Service Layer, mengelola local state (list todos, status isLoading, errorMessage), dan mengeksekusi logika bisnis tanpa menangani styling UI.
- **3. Service Layer (lib/services/todo_service.dart):** Mengelola seluruh komunikasi jaringan HTTP/REST API ke endpoint Supabase menggunakan Dio, menyematkan apikey dan Authorization token, serta melakukan serialisasi/deserialisasi JSON ke Model.
- **4. Domain / Model Layer (lib/models/todo_model.dart):** Mendefinisikan entitas data Todo (id, name, description, isDone) beserta method factory fromJson dan toJson.
- **5. Environment Config (lib/env/env_config.dart & assets/.env):** Menyimpan kredensial sensitif seperti Supabase Project URL dan Anon Key secara aman menggunakan library flutter_dotenv.

C. LANGKAH-LANGKAH PENGERJAAN & IMPLEMENTASI KODE

1. Konfigurasi Dependencies (pubspec.yaml)

Menambahkan library dio untuk HTTP client dan flutter_dotenv untuk konfigurasi file environment di pubspec.yaml:

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  dio: ^5.11.1
  flutter_dotenv: ^6.0.1

flutter:
  uses-material-design: true
  assets:
    - assets/.env
```

2. Konfigurasi Environment (assets/.env & lib/env/env_config.dart)

Menyimpan URL proyek Supabase dan Public Anon Key pada file assets/.env:

```
SUPABASE_URL=https://qjujyqvkrdqhbiqfdchg.supabase.co
SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_TODOS_TABLE=todos
```

Membaca konfigurasi environment secara terpusat pada file lib/env/env_config.dart:

```
import 'package:flutter_dotenv/flutter_dotenv.dart';

class EnvConfig {
  static String get supabaseUrl => dotenv.env['SUPABASE_URL'] ?? '';
  static String get supabaseAnonKey => dotenv.env['SUPABASE_ANON_KEY'] ?? '';
  static String get todosTable => dotenv.env['SUPABASE_TODOS_TABLE'] ?? 'todos';
}
```

3. Model Layer (lib/models/todo_model.dart)

Membuat representasi data todo beserta parsing JSON dari Supabase:

```
class TodoModel {
  final int id;
  final String name;
  final String description;
  final bool isDone;

  TodoModel({
    required this.id,
    required this.name,
    required this.description,
    required this.isDone,
  });

  factory TodoModel.fromJson(Map<String, dynamic> json) {
    return TodoModel(
      id: json['id'] as int,
      name: json['name'] ?? '',
      description: json['description'] ?? '',
      isDone: json['is_done'] as bool? ?? false,
    );
  }
```

```
  Map<String, dynamic> toJson() {
    return {
      'name': name,
      'description': description,
      'is_done': isDone,
    };
  }
}
```

4. Service Layer (lib/services/todo_service.dart)

Mengimplementasikan request HTTP REST API (GET, POST, PATCH, DELETE) menggunakan Dio:

```
import 'package:dio/dio.dart';
import '../env/env_config.dart';
import '../models/todo_model.dart';

class TodoApiService {
  final String _baseUrl = EnvConfig.supabaseUrl;
  final String _anonKey = EnvConfig.supabaseAnonKey;
  final String _table = EnvConfig.todosTable;

  late final Dio _dio;

  TodoApiService() {
    _dio = Dio(
      BaseOptions(
        baseUrl: _baseUrl,
        headers: {
          'apikey': _anonKey,
          'Authorization': 'Bearer $_anonKey',
          'Content-Type': 'application/json',
        },
      ),
    );
  }

  /// GET /rest/v1/todos?select=*
  Future<List<TodoModel>> fetchTodos() async {
    final response = await _dio.get(
      '/rest/v1/$_table',
      queryParameters: {'select': '*'},
    );
    final data = response.data as List<dynamic>;
    return data.map((e) => TodoModel.fromJson(e)).toList();
  }

  /// POST /rest/v1/todos
  Future<TodoModel> addTodo({required String name, required String description}) async {
    final response = await _dio.post(
      '/rest/v1/$_table',
      data: {'name': name, 'description': description, 'is_done': false},
      options: Options(headers: {'Prefer': 'return=representation'}),
    );
```

```
    final list = response.data as List<dynamic>;
    return TodoModel.fromJson(list.first);
  }

  /// PATCH /rest/v1/todos?id=eq.<id>
  Future<TodoModel> toggleTodo(int id, bool isDone) async {
    final response = await _dio.patch(
      '/rest/v1/$_table',
      queryParameters: {'id': 'eq.$id'},
      data: {'is_done': isDone},
      options: Options(headers: {'Prefer': 'return=representation'}),
    );
    final list = response.data as List<dynamic>;
    return TodoModel.fromJson(list.first);
  }

  /// DELETE /rest/v1/todos?id=eq.<id>
  Future<void> deleteTodo(int id) async {
    await _dio.delete(
      '/rest/v1/$_table',
      queryParameters: {'id': 'eq.$id'},
    );
  }
}
```

5. Controller Layer (lib/controllers/todo_controller.dart)

Mengatur state lokal aplikasi (loading, error, dan list data):

```
import '../models/todo_model.dart';
import '../services/todo_service.dart';

class TodoController {
  final TodoApiService _service;
  TodoController(this._service);

  List<TodoModel> todos = [];
  bool isLoading = false;
  String? errorMessage;

  Future<void> loadTodos() async {
    isLoading = true;
    errorMessage = null;
    try {
      todos = await _service.fetchTodos();
    } catch (e) {
      errorMessage = e.toString();
    } finally {
      isLoading = false;
    }
  }

  Future<void> addTodo(String name, String description) async {
    isLoading = true;
    try {
```

```
      final newTodo = await _service.addTodo(name: name, description: description);
      todos.add(newTodo);
    } catch (e) {
      errorMessage = e.toString();
    } finally {
      isLoading = false;
    }
  }

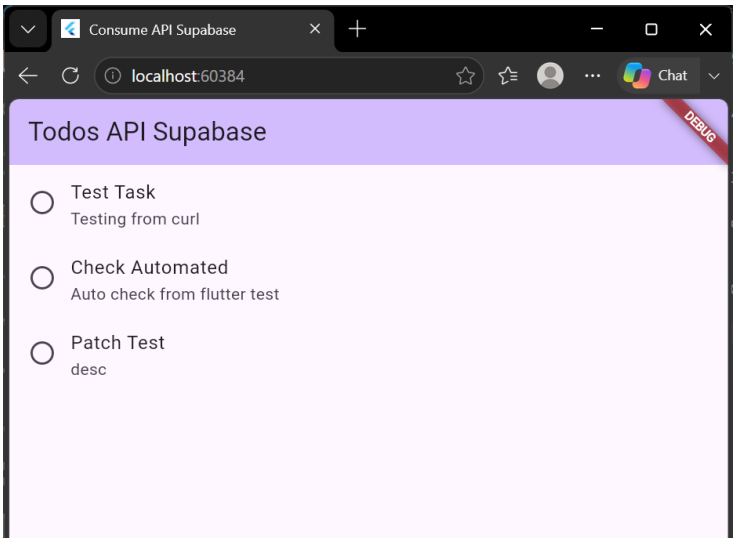
  Future<void> toggleTodoDone(TodoModel todo) async {
    isLoading = true;
    try {
      final updated = await _service.toggleTodo(todo.id, !todo.isDone);
      final index = todos.indexWhere((t) => t.id == todo.id);
      if (index != -1) todos[index] = updated;
    } catch (e) {
      errorMessage = e.toString();
    } finally {
      isLoading = false;
    }
  }

  Future<void> deleteTodo(TodoModel todo) async {
    isLoading = true;
    try {
      await _service.deleteTodo(todo.id);
      todos.removeWhere((t) => t.id == todo.id);
    } catch (e) {
      errorMessage = e.toString();
    } finally {
      isLoading = false;
    }
  }
}
```

D. HASIL DAN SCREENSHOT PENGUJIAN SISTEM

1. Pengujian Read / Fetch Data dari Supabase (GET)

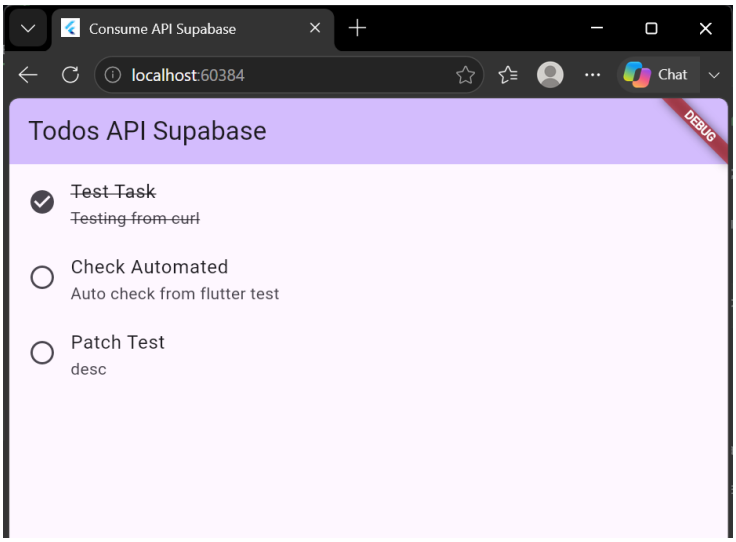
Aplikasi dijalankan pada browser lokal (Flutter Web). Saat initState() dieksekusi, controller memanggil fetchTodos() ke endpoint REST Supabase. Data yang tersimpan di tabel PostgreSQL berhasil diambil dan dirender secara dinamis ke dalam ListView:



Gambar 1: Tampilan Read / Fetch Data Todo dari Supabase (GET)

2. Pengujian Update / Toggle Status Selesai (PATCH)

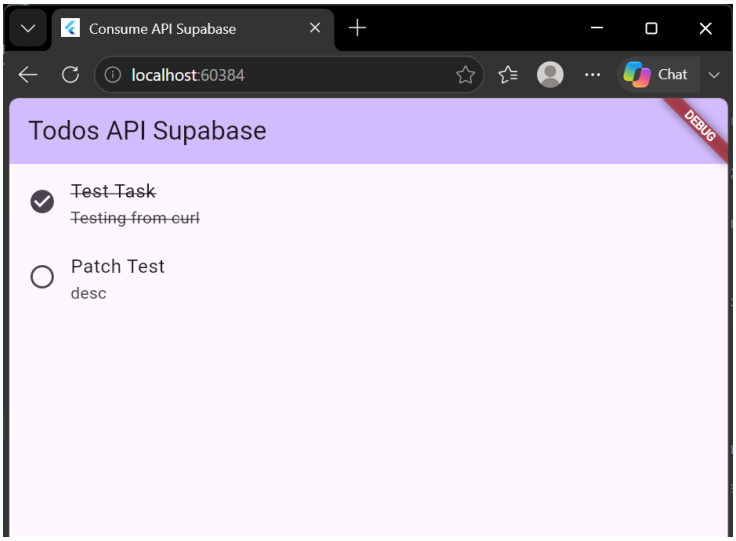
Ketika pengguna melakukan tap pada salah satu todo ('Test Task'), controller mengirimkan request PATCH dengan parameter query id=eq.1 dan payload data {'is_done': true}. Status berhasil diperbarui di database Supabase, sehingga tampilan UI seketika berubah menampilkan ikon lingkaran centang dan efek coretan (strikethrough):



Gambar 2: Pengujian Update / Toggle Status Selesai Todo (PATCH)

3. Pengujian Delete Data Todo (DELETE)

Ketika pengguna memilih aksi 'Delete' pada item 'Check Automated', aplikasi mengirimkan HTTP DELETE ke Supabase dengan filter query id=eq.<id>. Supabase mengembalikan kode status 204 No Content, dan controller memperbarui state lokal dengan menghapus item tersebut dari daftar tampilan:



Gambar 3: Pengujian Delete Data Todo dari Database Supabase (DELETE)

E. KESIMPULAN

Berdasarkan praktikum Week 09 Data Storage (1) yang telah diselesaikan:

- Penerapan arsitektur Simplified MVC dengan Service Layer terbukti mempermudah pengelolaan kode, di mana tanggung jawab antara UI, State Management, dan Jaringan HTTP terpisah secara bersih dan modular.
- Supabase PostgREST API mempermudah integrasi database PostgreSQL ke dalam aplikasi mobile Flutter tanpa perlu membuat server backend custom secara manual.
- Pengujian fungsionalitas CRUD (Create, Read, Update, Delete) berhasil dilakukan secara menyeluruh dengan status koneksi database yang stabil dan responsif.